

Documentation technique

Data Lake financier

Groupe : Artemiy Smogunov, Nathan Smadja-Tubiana, Patrice Ignongui

Branche : improve/assignment-compliance

Généré le : 2026-07-10 12:58 UTC

Data Lake financier – Démo technique

Source:

/home/ubuntu/work/Projet-DataLake-Final/README.md

Financial Data Lake (Projet EFREI 2025-2026)

Projet final du cours *Data Lakes & Data Integration*.

Groupe

- Artemiy Smogunov
- Nathan Smadja-Tubiana
- Patrice Ignongui

Lien du sujet

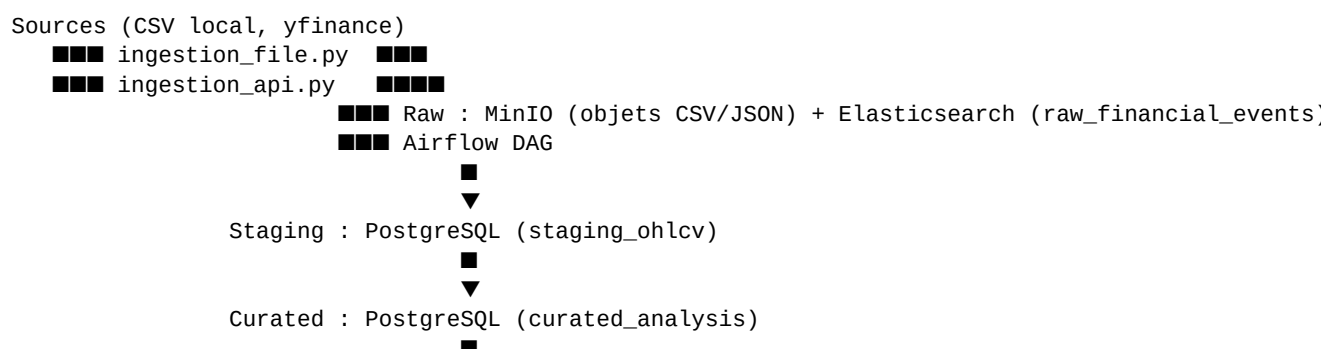
- Sujet : [Data_Lakes_Projet_Final_EFREI_2025-2026.pdf](consignes/Data_Lakes_Projet_Final_EFREI_2025-2026.pdf)
- Récapitulatif des règles : [resume_regles_devoir.txt](consignes/resume_regles_devoir.txt)

Ce que couvre ce dépôt

Ce projet met en place un mini data lake financier avec deux sources de données, trois zones (Raw / Staging / Curated), une API d'exposition, et une orchestration planifiée.

- **Sources utilisées**
 - data/finance_dataset.csv (fichier local)
 - API Yahoo Finance via yfinance
- **Zone Raw** : MinIO (S3 compatible) + Elasticsearch
- **Zone Staging + Zone Curated** : PostgreSQL
- **Orchestration** : Apache Airflow
- **Gateway API** : FastAPI

Architecture (vue courte)



▼
FastAPI : /raw, /staging, /curated, /health, /stats
■
■■■ endpoints avancés /ingest, /ingest_fast

Correspondance avec le devoir

| Exigence | Mise en œuvre | Preuve principale |

|---|---|---|

| Conception d'un data lake complet | ingestion/, transformation/, api/, airflow/ | README + code + scripts SQL |

| Structure Raw / Staging / Curated | ingestion/, transformation/staging/, transformation/curated/ | scripts/init_db.sql, requêtes de api/routers |

| Stockage raw compatible S3 + Elasticsearch | ingestion/ingest_file.py, ingestion/ingest_api.py, docker-compose.yml, api/dependencies.py | index/métadonnées dans ES, objets MinIO |

| Ingestion depuis 2 sources | ingestion/ingest_file.py, ingestion/ingest_api.py, config/settings.py | ALL_TICKERS + pipeline d'ingestion |

| Pipeline d'intégration | airflow/dags/financial_pipeline_dag.py | tâches parallèle file/API puis staging puis curated |

| API Gateway obligatoire | api/main.py, api/routers/* | endpoints /raw, /staging, /curated, /health, /stats |

| Niveau avancé /ingest + /ingest_fast + benchmark | api/routers/ingest.py, api/routers/ingest_fast.py, scripts/benchmark_endpoints.py | script de benchmark reproductible |

Stack et ports

| Élément | Rôle | Port | Lieu |

|---|---|---|---|

| PostgreSQL | Staging, Curated, logs d'ingestion | 5432 | docker-compose.yml |

| MinIO API | Stockage objets Raw | 9000 | docker-compose.yml |

| MinIO Console | UI de buckets | 9001 | docker-compose.yml |

| Elasticsearch | Index brut | 9200 | docker-compose.yml |

| Airflow UI | Orchestration DAG | 8080 | airflow services |

| FastAPI | API Gateway | 8000 | api/ service |

| Redis | Service vérifié par /health | 6379 | api/dependencies.py |

Prérequis

- Docker + Docker Compose
- Git (pour la reproduction du dépôt)

Installation et lancement

```
git clone https://github.com/Nathan2412/Projet-DataLake-Final.git
cd Projet-DataLake-Final
```

```
docker compose up -d
```

Attendre le démarrage puis vérifier l'état :

```
docker compose ps
docker compose logs api
```

Services attendus : PostgreSQL, MinIO, Elasticsearch, Airflow Webserver, Airflow Scheduler, API.

Vérification rapide après démarrage

```
curl http://localhost:8000/health
curl http://localhost:8000/docs
curl http://localhost:8080
curl http://localhost:9001
```

Endpoints attendus

- GET /health — vérifie Postgres, MinIO, Elasticsearch, Redis.
- GET /stats — métriques par zone.
- GET /raw — documents bruts depuis Elasticsearch avec filtres (ticker, source, dates, limit).
- GET /raw/objects — objets présents dans MinIO.
- GET /staging — OHLCV nettoyé + indicateurs.
- GET /staging/tickers — liste des tickers disponibles.
- GET /curated — anomalies + signaux de trading.
- GET /curated/anomalies/summary — résumé des anomalies par ticker/type.
- GET /curated/signals — tickers avec signal actif buy ou sell.
- POST /ingest — pipeline standard (séquentiel).
- POST /ingest_fast — pipeline avancé (optimisé).

Appels d'ingestion utiles

Ingestion standard

```
curl -X POST http://localhost:8000/ingest \
-H "Content-Type: application/json" \
-d '{
  "data": {
    "tickers": ["AAPL", "MSFT", "^GSPC"],
    "period": "1mo",
    "run_staging": true,
    "run_curated": true
  }
}'
```

Ingestion optimisée

```
curl -X POST http://localhost:8000/ingest_fast \
-H "Content-Type: application/json" \
-d '{
  "data": {
    "tickers": ["AAPL", "MSFT", "^GSPC"],
    "period": "1mo",
    "run_staging": true,
    "run_curated": true
  }
}'
```

Les deux endpoints retournent une structure avec status, pipeline_steps, performance, errors, timestamp.

Comment vérifier les données manuellement

```
curl "http://localhost:8000/staging?ticker=AAPL&limit=20"
curl "http://localhost:8000/curated?ticker=AAPL&anomalies_only=true&limit=20"
curl "http://localhost:8000/stats"
```

Tests exécutables localement (sans stack externe)

```
python -m pytest tests/test_financial_logic.py tests/test_benchmark.py
```

Résultat observé sur cette branche : tests passants en local (hors services).

Benchmark reproductible

Le script enregistre systématiquement un rapport JSON, pour batch 1 et 100.

```
python scripts/benchmark_endpoints.py \
--base-url http://localhost:8000 \
--period 5d \
--repeats 3 \
--timeout 1800 \
--output livrables/benchmark_run.json
```

Interprétation rapide du rapport (livrables/benchmark_run.json) :

- chaque result contient un bloc batch_size (1 puis 100), standard_wall_ms_median, fast_wall_ms_median, gain_pct_median, target_gain_pct et target_met.
- un gain cible target_gain_pct = 30 est attendu par le sujet.

Ne pas considérer une valeur fixe comme preuve définitive. **Le benchmark doit être relancé sur Docker Host après chaque changement de branche** et interprété avec la version de la branche courante.

Limites connues

- La preuve complète de bout-en-bout dépend de services Docker actifs (docker -compose).

- Les temps d'ingestion varient selon la disponibilité réseau Yahoo Finance.
- Les données testées ci-dessus sont des données courtes de requête pour un run court ; on doit refaire la boucle complète en environnement réel pour une validation finale.
- Le vieux fichier `livrables/benchmark_ingest_vs_ingest_fast.json` est conservé comme historique, et n'est pas une preuve de réexécution de la branche actuelle.

Régénérer les PDF

```
.venv/bin/pip install -r scripts/requirements-docs.txt  
.venv/bin/python scripts/generate_pdf_deliverables.py
```

Livrables attendus

- `livrables/Rapport_DataLake_Finance_Artemiy_Smogunov_Nathan_Smadja-Tubiana_Patrice_Ignongui.pdf`
- `livrables/Documentation_Technique_DataLake_Finance_Artemiy_Smogunov_Nathan_Smadja-Tubiana_Patrice_Ignongui.pdf`
- `livrables/benchmark_ingest_vs_ingest_fast.json` (historique)

Source: /home/ubuntu/work/Projet-DataLake-Final/docs/CONFORMITE_DEVOIR.md

Conformité du devoir

Version et état

- Sujet : EFREI Data Lakes & Data Integration 2025-2026
- Branche : `improve/assignment-compliance`

Ce tableau est le suivi des exigences du sujet, avec ce que le code met en place aujourd'hui et la façon de le vérifier.

- **implemented** = livrable/procédure présent dans le dépôt et testable.
- **needs live rerun** = la preuve dépend d'une exécution Docker/API réelle (à relancer sur hôte Docker).

Matrice de conformité

| # | Exigence | Type | Fichiers d'implémentation | Vérification | Statut |

|---|---|---|---|---|---|

| 1 | Concevoir et implémenter un Data Lake de bout en bout (ingestion, stockage, transformation, exposition) | Obligatoire | `ingestion/*`, `transformation/*`, `api/*`, `airflow/dags/financial_pipeline_dag.py`, `scripts/init_db.sql` | `python -m pytest tests/test_financial_logic.py tests/test_benchmark.py` (couverture structurelle), puis `docker compose up -d + GET /health, GET /raw, /staging, /curated` | implemented |

| 2 | Structurer le Data Lake en zones Raw / Staging / Curated | Obligatoire | `ingestion/ingest_file.py`, `ingestion/ingest_api.py`, `transformation/staging/transform_staging.py`, `transformation/curated/transform_curated.py`, `docker-compose.yml` | `python scripts/benchmark_endpoints.py --base-url http://localhost:8000 --repeats 3` (après démarrage) | implemented |

| 3 | Utiliser Elasticsearch ou stockage type S3 pour Raw | Obligatoire | `docker-compose.yml`, `ingestion/ingest_file.py`, `ingestion/ingest_api.py`, `api/dependencies.py`, `api/routers/raw.py` | Démarrage Docker + `GET /raw`, `GET /stats` (sections `raw_elasticsearch`, `raw_minio`) | implemented |

| 4 | Utiliser deux sources de données | Obligatoire | `ingestion/ingest_file.py`, `ingestion/ingest_api.py`, `config/settings.py` | Tests des identifiants par source, puis `GET /raw?source=yfinance_file` et `/raw?source=yfinance_api` sur la stack | implemented |

| 5 | Mettre en place une pipeline d'intégration avec Airflow | Obligatoire | `airflow/dags/financial_pipeline_dag.py`, `ingestion/ingest_file.py`, `ingestion/ingest_api.py`, `transformation/staging/transform_staging.py`, `transformation/curated/transform_curated.py` | Démarrage `docker compose up -d`, check Airflow UI `http://localhost:8080`, DAG `financial_data_lake_pipeline` | needs live rerun |

| 6 | Exposer API Gateway avec /raw, /staging, /curated, /health, /stats | Obligatoire |
api/main.py, api/routers/raw.py, api/routers/staging.py, api/routers/curated.py,
api/routers/health.py, api/routers/stats.py | curl
http://localhost:8000/{health,stats,raw,staging,curated} (avec params adaptés) |
implemented |

| 7 | Niveau avancé : POST /ingest | Optionnel | api/routers/ingest.py | curl -X POST
http://localhost:8000/ingest ... | implemented |

| 8 | Niveau avancé : POST /ingest_fast | Optionnel | api/routers/ingest_fast.py | curl
-X POST http://localhost:8000/ingest_fast ... | implemented |

| 9 | Niveau avancé : benchmark batch 1 et 100 | Optionnel | scripts/benchmark_endpoints.py |
python scripts/benchmark_endpoints.py --base-url http://localhost:8000
--period 5d --repeats 3 --output ... | needs live rerun |

| 10 | Niveau avancé : amélioration > 30% | Optionnel | scripts/benchmark_endpoints.py,
api/routers/ingest.py, api/routers/ingest_fast.py | Vérifier dans le rapport JSON
target_met = true après exécution sur branche courante | needs live rerun |

| 11 | Livrables complets | Obligatoire | README.md, livrables/RAPPORT_TECHNIQUE.md, PDF à
régénérer après validation | Vérifier les fichiers et leurs liens | needs live rerun |

Notes de preuve

- Les tests unitaires ne valident pas l'exécution complète des services.
- La preuve finale attendue est : "tests off-line passants + exécution Docker + endpoints + benchmark rerunnable".
- Le fichier livrables/benchmark_ingest_vs_ingest_fast.json est historique (mesures passées) et n'est pas une preuve de la branche actuelle.